LAPORAN PRAKTIKUM 6

SEARCHING

NAMA : FADILA MAYAMASITA

NIM : 254107020012

KELAS : 1C TEKNIK INFORMATIKA

D – IV TEKNIK INFORMATIKA

TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG 2026

6.2.1. Langkah-langkah Percobaan Sequential Search

1. Pada pertemuan jobsheet 6 ini akan menggunakan class Mahasiswa, MahasiswaBerprestasi, dan MahasiswaDemo pada pertemuan jobsheet 5 sebelumnya, kemudian buka ke tiga class tersebut di text editor.

2. Tambahkan method sequentialSearching bertipe integer dengan parameter cari bertipe double pada class MahasiswaBerprestasi. Kemudian Deklarasikan isi method sequentialSearching dengan algoritma pencarian data menggunakan teknik sequential searching.

Jawaban:

```java
package MINGGU6;

public class MahasiswaBerprestasi13 {
    Mahasiswa13 [] listMhs= new Mahasiswa13[5];
    int idx;

    int sequentialSearching(double cari){
        int posisi = -1;
        for (int j=0; j<listMhs.length; j++){
            if (listMhs[j].ipk==cari){
                posisi=j;
                break;
            }
        }
        return posisi;
    }
```

3. Buatlah method tampilPoisisi bertipe void dan Deklarasikan isi dari method tampilPoisisi pada class MahasiswaBerprestasi.

Jawaban:

```java
void tampilPosisi(double x, int pos) {
    if (pos!=-1) {
        System.out.println("Data Mahasiswa dengan IPK :" +x+ " ditemukan pada indeks " +pos);
    }
    else {
        System.out.println("data " +x+ "tidak ditemukan");
    }
}
```

4. Pada class MahasiswaBerprestasi, buatlah method tampilDataSearch bertipe void dan deklarasikan isi dari method tampilDataSearch.

Jawaban:

```
}
void tampilDataSearch(double x, int pos) {
    if (Pos !=-1) {
        System.out.println("nim\t : "+listMhs[pos].nim);
        System.out.println("nama\t :"+listMhs[pos].nama);
        System.out.println("kelas\t :"+listMhs[pos].kelas);
        System.out.println("ipk\t :"+listMhs[pos].ipk);
    }
    else {
        System.out.println("Data Mahasiswa dengan IPK " +x+ "tidak ditemukan ");
    }
}
```

5. Pada class MahasiswaDemo, tambahkan kode program berikut ini untuk melakukan pencarian data dengan algoritma sequential searching.

Jawaban:

```
list.tampil();
//melakukan percobaan data sequential
System.out.println(x: "-----------------------------------------------");
System.out.println(x: "Pencari data");
System.out.println(x: "-----------------------------------------------");
System.out.println(x: "masukkan ipkp mahasiswa yang dicari: ");
System.out.print(s: "IPK: ");
double cari = sc.nextDouble();

System.out.println(x: "Menggunakan sequential searching");
double posisi = list.sequentialSearching(cari);
int pss = (int)posisi;
list.tampilPosisi(cari, pss);
list.tampilDataSearch(cari, pss);
```

6. Jalankan dan amati hasilnya.

Jawaban:

```
3f5fb86eb\redhat.java\jdt_ws\Algoritma-stru
Masukkan data Mahasiswa ke-1
--------------------------------------
NIM : 111
Nama  : adi
Kelas : 2
IPK  : 3,6

Masukkan data Mahasiswa ke-2
--------------------------------------
NIM : 222
Nama  : tio
Kelas : 2
IPK  : 3,8

Masukkan data Mahasiswa ke-3
--------------------------------------
NIM : 333
Nama  : ila
Kelas : 2
IPK  : 3,0

Masukkan data Mahasiswa ke-4
--------------------------------------
NIM : 444
Nama  : lia
Kelas : 2
IPK  : 3,5

Masukkan data Mahasiswa ke-5
--------------------------------------
NIM : 555
Nama  : fia
Kelas : 2
IPK  : 3,3

Nama: adi
NIM: 111
Kelas: 2
IPK: 3.6
```

```
--------------------------------------------------
Pencari data
--------------------------------------------------
masukkan ipk mahasiswa yang dicari:
IPK: 3,5
Menggunakan sequential searching
Data Mahasiswa dengan IPK :3.5 ditemukan pada indeks 3
nim      : 444
nama     :lia
kelas    :2
ipk      :3.5
```

6.2.3. Pertanyaan

1. Jelaskan perbedaan metod tampilDataSearch dan tampilPosisi pada class MahasiswaBerprestasi!

Jawaban:

tampilPosisi: Berfungsi hanya untuk memberikan konfirmasi apakah data ada atau tidak. Jika ada, method ini hanya memberi tahu **lokasi (indeks)** di mana data tersebut disimpan dalam array.

tampilDataSearch: Berfungsi untuk menyajikan **informasi lengkap** dari objek mahasiswa yang ditemukan. Method ini tidak hanya memberi tahu bahwa data ada, tapi langsung menampilkan detail atributnya (NIM, Nama, Kelas, IPK).

2. Jelaskan fungsi break pada kode program di bawah ini!

Jawaban:

Fungsi break pada kode tersebut berfungsi untuk menghentikan proses perulangan (*looping*) secara paksa segera setelah kondisi data yang dicari (IPK) ditemukan, sehingga program tidak perlu lagi memeriksa sisa elemen array yang ada. Penggunaan break ini bertujuan untuk meningkatkan efisiensi eksekusi program dengan menghemat waktu komputasi, serta memastikan bahwa variabel posisi menyimpan indeks dari data pertama yang berhasil ditemukan sebelum keluar dari blok perulangan tersebut.

3.Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?

Jawaban:

Variabel pos (atau posisi) berfungsi sebagai penanda status dan lokasi data dalam array. Secara spesifik, variabel ini menyimpan nilai indeks tempat data ditemukan sehingga program dapat mengakses kembali atribut objek tersebut (seperti nama atau NIM) di luar blok perulangan. Selain itu, nilai awal yang biasanya diatur ke -1 berfungsi sebagai indikator logika: jika setelah perulangan nilai pos tetap -1, maka program dapat menyimpulkan bahwa data tidak ditemukan, namun jika nilai berubah menjadi positif, itu menjadi bukti bahwa data tersedia di lokasi tersebut.

4.Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan.

Jawaban:

Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian yang ditampilkan adalah data pertama yang ditemukan (indeks terkecil). Hal ini terjadi karena adanya perintah break di dalam blok if (listMhs[j].ipk == cari). Begitu program menemukan kecocokan untuk pertama kalinya, perintah break akan langsung menghentikan seluruh proses perulangan, sehingga program tidak sempat memeriksa elemen-elemen setelahnya dan langsung mengunci nilai indeks pertama tersebut ke dalam variabel posisi.

5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah break dihapus dari kode di atas?

Jawaban:

Jika perintah break dihapus dari kode tersebut, maka perulangan (*looping*) tidak akan berhenti meskipun data yang dicari sudah ditemukan. Program akan terus memindai seluruh elemen array hingga indeks terakhir.

6.3. Searching/ Pencarian Menggunakan Algoritma Binary Search

6.3.1. Langkah-langkah Percobaan Binary Search

1. Pada percobaan 6.2.1 (sequential search) tambahkan method findBinarySearch bertipe integer pada class MahasiswaBerprestasi. Kemudian Deklarasikan isi method findBinarySearch dengan algoritma pencarian data menggunakan teknik binary searching.

Jawaban:

```
int findBinarySearch(double cari, int left, int right){
    int mid;
    if (right>=left){
        mid =(left+right)/2;
        if (cari ==listMhs[mid].ipk){
            return (mid);
        }
        else if (listMhs[mid].ipk>cari){
            return findBinarySearch(cari, left, mid-1);
        }
        else{
            return findBinarySearch(cari, left, right);
        }
    }
    return -1;
}
```

2. Panggil method findBinarySearch terdapat pada class MahasiswaBerprestasi di kelas MahasiswaDemo. Kemudia panggil method tampilPosisi dan tampilDataSearch

Jawaban:

```
//Melakukan pencarian data Binary
System.out.println(x: "-----------------------------------------------");
System.out.println(x: "Pencarian data");
System.out.println(x: "-----------------------------------------------");
System.out.println(x: "masukkan IPK mahasiswa yang dicari: ");
System.out.print(s: "IPK: ");
cari = sc.nextDouble();
System.out.println(x: "-----------------------------------------------");
System.out.println(x: "Menggunaakan binary search");
double posisi2 = list.findBinarySearch(cari, left: 0, jmlMhs-1);
int pss2= (int)posisi2;
list.tampilPosisi(cari, pss2);
list.tampilDataSearch(cari, pss2);
```

3. Jalankan dan amati hasilnya (inputkan data IPK secara terurut -ASC seperti verifikasi hasil percobaan di bawah ini).

Jawaban:

```
Nama: adi
NIM: 111
Kelas: 2
IPK: 3.1
--------------------------------
Nama: ila
NIM: 222
Kelas: 2
IPK: 3.2
--------------------------------
Nama: lia
NIM: 333
Kelas: 2
IPK: 3.3
--------------------------------
Nama: susi
NIM: 444
Kelas: 2
IPK: 3.5
--------------------------------
Nama: anita
NIM: 555
Kelas: 2
IPK: 3.7
--------------------------------
-----------------------------------------------
Pencari data
-----------------------------------------------
masukkan ipk mahasiswa yang dicari:
IPK: 3,7
Menggunakan sequential searching
Data Mahasiswa dengan IPK :3.7 ditemukan pada indeks 4
nim      : 555
nama     :anita
kelas    :2
ipk      :3.7
-----------------------------------------------
```

6.3.3. Pertanyaan

1.Tunjukkan pada kode program yang mana proses divide dijalankan!

Jawaban:

Proses *divide* terjadi pada baris kode: mid = (left + right) / 2;. Di sini, rentang pencarian dibagi menjadi dua bagian dengan menentukan titik tengahnya.

2.Tunjukkan pada kode program yang mana proses conquer dijalankan!

Jawaban:

Proses *conquer* terjadi saat program melakukan pemanggilan rekursif untuk memperkecil jangkauan pencarian, yaitu pada baris:

- return findBinarySearch(cari, left, mid - 1); (mencari di bagian kiri).
- return findBinarySearch(cari, mid + 1, right); (mencari di bagian kanan).

3. Apa fungsi left, right, dan mid?

Jawaban:

left: Menandai batas awal (indeks terendah) dari rentang pencarian yang aktif.

right: Menandai batas akhir (indeks tertinggi) dari rentang pencarian yang aktif.

mid: Menandai nilai tengah antara left dan right yang digunakan sebagai acuan perbandingan dengan nilai yang dicari.

4. Jika data IPK yang dimasukkan tidak urut. Apakah program masih dapat berjalan? Mengapa demikian?

Jawaban:

Program mungkin **tidak dapat menemukan data** atau memberikan hasil yang salah. Syarat mutlak algoritma *Binary Search* adalah data harus dalam keadaan terurut agar logika pembagian rentang (lebih besar/lebih kecil) dapat berfungsi dengan benar.

5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program binary seach agar hasilnya sesuai

Jawaban:

Hasilnya tidak akan sesuai dengan kode asli karena logika perbandingannya dibuat untuk data menaik (*Ascending*).Modifikasi agar sesuai: Ubah kondisi perbandingan. Jika listMhs[mid].ipk < cari, maka cari ke arah kiri (mid - 1). Jika listMhs[mid].ipk > cari, maka cari ke arah kanan (mid + 1).

6. Jelaskan bagaimana binary search menentukan bahwa data yang dicari tidak ditemukan di dalam array.

Jawaban:

Binary search menentukan data tidak ditemukan jika nilai right sudah lebih kecil dari left (right < left). Pada kondisi ini, fungsi akan mengembalikan nilai -1.

7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.

Jawaban:

```
// constructor
MahasiswaBerprestasi13(int jmlMhs) {
    listMhs = new Mahasiswa13[jmlMhs];
    idx = 0;
}

ublic static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println(x: "Masukkan Jumlah maahasiswa: ");
int jmlMhs=sc.nextInt();
sc.nextLine();
```